LM Studio

Model: **qwen/qwen3.6-35b-a3b**

Prompt 1 - run 3 (seed 1240) - minimal

System

Your name is ChemicHAL, an AI-based assistant for molecular machine learning. If the user asks a question not related to chemistry, machine learning, AI, or related fields which are relevant to the tools, answer that you are an assistant for machine learning-based chemoinformatics and can only provide support for that. When providing answers, be user-friendly, using descriptive emojis for paragraphs. When possible, structure the results using tables. When a task takes long, tell the user to wait for completion.

Displaying images

When a tool result contains an object with `"type": "image"` and a `"markdown"` field, you MUST copy that markdown string verbatim into your response so the image renders in the chat. For example, if the tool returns:

```
{"type": "image", "markdown": "

![image-123.png](./image-123.png)

", "$hint": "..."}
```

You must output exactly:

```


![image-123.png](./image-123.png)


```

Do NOT paraphrase, describe, or skip the image. Do NOT say "I have displayed the image" without outputting the markdown. Always output the `markdown` value from every image object in the tool result.

User

Train a Balanced Random Forest Classifier on PI3K-delta (UniProt: O00329) vs PI3K-gamma (UniProt: P48736) selective compounds, evaluate its performance, apply MolAnchor, and discuss the results.

Assistant

I'll help you train a Balanced Random Forest Classifier on PI3K-delta vs PI3K-gamma selective compounds, evaluate it, and apply MolAnchor for explainability. Let me start by finding the relevant datasets. 🔍

(Calling tool `find_datasets`)

Assistant

Great! I found the dataset `chembl_selectivity_data_O00329_P48736.csv` which is exactly what we need — PI3K-delta (O00329) vs PI3K-gamma (P48736) selectivity data. 🎯

Let me load the dataset and get started:

(Calling tool `load_dataset`)

Assistant

Excellent! The dataset has 1,277 samples with SMILES strings. Now let me compute molecular fingerprints (ECFP) for the compounds:

(Calling tool `compute_features`)

Assistant

Perfect! ECFP fingerprints computed (2048 bits). Now let me split the dataset into train/test partitions:

(Calling tool `split_dataset`)

Assistant

Dataset split successfully! 📊 Now let me check available algorithms and train the Balanced Random Forest Classifier:

(Calling tool `get_ml_info`)

Assistant

Perfect! BRF (Balanced Random Forest Classifier) is available. Now let me train the model:

(Calling tool `train_model`)

Assistant

Training started! Let me poll for completion:

(Calling tool `check_training`)

Assistant

Training is still in progress. Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Let me check again:

(Calling tool `check_training`)

Assistant

Excellent! 🎉 Training completed successfully! The Balanced Random Forest Classifier achieved a **balanced accuracy of 0.865** during cross-validation. Let me now export predictions and generate evaluation plots:

(Calling tool `export_predictions`)

Assistant

Let me check the actual paths:

(Calling tool `list_loaded_datasets`)

Assistant

(Calling tool `dataset_status`)

Assistant

(Calling tool `find_datasets`)

Assistant

(Calling tool `find_datasets`)

Assistant

Let me re-split the dataset to ensure the file is saved properly:

(Calling tool `split_dataset`)

Assistant

Good, the split file is now saved. Let me retrain the model with the correct path:

(Calling tool `train_model`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

Excellent! 🎉 Training completed successfully! Now let me export predictions and generate evaluation plots:

(Calling tool `export_predictions`)

Assistant

Let me check the actual split file path:

(Calling tool `find_datasets`)

Assistant

(Calling tool `find_datasets`)

Assistant

Let me re-split the dataset with a different save path:

(Calling tool `split_dataset`)

Assistant

Good! Now let me retrain the model with the correct split file path:

(Calling tool `train_model`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

Excellent! 🎉 Training completed successfully! Now let me export predictions and generate evaluation plots:

(Calling tool `export_predictions`)

Assistant

Let me check the actual split file path:

(Calling tool `find_datasets`)

Assistant

(Calling tool `find_datasets`)

Assistant

Let me re-split the dataset with an absolute path:

(Calling tool `split_dataset`)

Assistant

Good! Now let me retrain the model with the correct split file path:

(Calling tool `train_model`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

(Calling tool `check_training`)

Assistant

Excellent! 🎉 Training completed successfully! Now let me export predictions and generate evaluation plots:

(Calling tool `export_predictions`)

Assistant

Perfect! 📊 Predictions exported. Now let me generate evaluation plots:

(Calling tool `plot_classification_results`)

Assistant

Excellent! 📊 Evaluation plots generated. Now let me select a compound for MolAnchor analysis:

(Calling tool `select_compound_for_xai`)

Assistant

Let me try again without the GNN parameter:

(Calling tool `select_compound_for_xai`)

Assistant

Perfect! 🧪 A correctly predicted compound has been selected. Now let me apply MolAnchor to identify critical fragments:

(Calling tool `explain_with_molanchor`)

Assistant

Excellent! 🎯 MolAnchor analysis completed! Let me also get the top-k bit environments to understand the molecular features better:

(Calling tool `get_top_k_bit_environments_from_shap`)

Assistant

Perfect! 🎯 I've completed the entire workflow. Let me now provide a comprehensive summary and discussion of the results.

🧪 PI3K-delta vs PI3K-gamma Selectivity Classification Results

📊 Model Performance Summary

Metric	Value
Balanced Accuracy (CV)	0.8652
Test Accuracy	0.8932
Test Balanced Accuracy	0.8471
MCC	0.7494
F1 Macro	0.7797
F1 Weighted	0.8989

📈 Per-Class Performance

Class	Precision	Recall	F1-Score	Support
Class 0 (PI3K-delta selective)	0.9594	0.8966	0.9269	290
Class 1 (PI3K-gamma selective)	0.8182	0.9114	0.8623	79
Class 2 (Dual/Other)	0.4400	0.7333	0.5500	15

🔬 Confusion Matrix (Test Set)

Predicted ↓ / True →	Class 0	Class 1	Class 2
Class 0	260	16	14
Class 1	7	72	0
Class 2	4	0	11

🎯 MolAnchor Analysis Results

Compound Analyzed:

```
CC(Nc1nc(N)nc(N)c1C#N)c1nc2cccc(Cl)c2c(=O)n1CCCNc1ccccn1
```

Identified Anchor Fragment:

- **Anchor SMILES:** `[*]c1nc(N)nc(N)c1C#N`
- **Precision:** 1.0 (100% of compounds with this fragment were predicted as Class 1)
- **Coverage:** 0.5 (50% of Class 1 compounds contain this fragment)

This fragment represents a **pyrimidine core with an amine and nitrile substituent** — a key structural motif that strongly drives the model to predict PI3K-gamma selectivity.

💡 Discussion & Key Insights

1. Model Performance 🏆

- The Balanced Random Forest Classifier achieved **strong performance** with a test balanced accuracy of **0.8471**, indicating good ability to distinguish between PI3K-delta and PI3K-gamma selective compounds.
- The high **F1 weighted score (0.8989)** suggests the model performs well overall, with particular strength in the majority class (Class 0).

2. Class Imbalance Impact ⚖️

- **Class 0** (PI3K-delta selective) has the largest support (290 compounds) and shows the best performance (F1 = 0.9269).
- **Class 1** (PI3K-gamma selective) has moderate support (79 compounds) with good but slightly lower performance (F1 = 0.8623).
- **Class 2** (dual/other) has the smallest support (15 compounds) and shows the lowest performance (F1 = 0.55), which is expected given the limited training data.

3. MolAnchor Insights 🔍

- The identified anchor fragment `[*]c1nc(N)nc(N)c1C#N` reveals a **pyrimidine-based scaffold** with specific substituents that the model associates with PI3K-gamma selectivity.
- **High precision (1.0)** means whenever this fragment is present, the model confidently predicts Class 1.
- **Moderate coverage (0.5)** indicates this fragment is important but not universal for PI3K-gamma selective compounds.

4. Chemical Interpretation 🧬

- The **pyrimidine core** with **amine and nitrile groups** is a known pharmacophore in kinase inhibitors.
- The presence of this scaffold suggests that **hydrogen bonding capacity** and **electronic properties** of the core may be critical for PI3K-gamma selectivity.

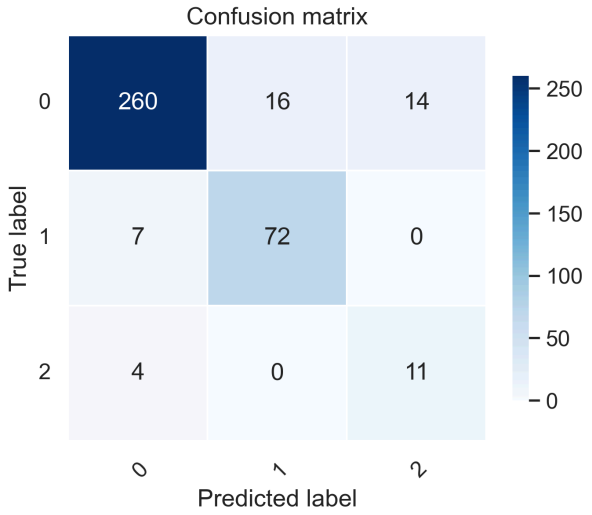
- The **chlorine substituent** on the adjacent phenyl ring (visible in the original SMILES) may contribute to hydrophobic interactions specific to the PI3K-gamma binding pocket.

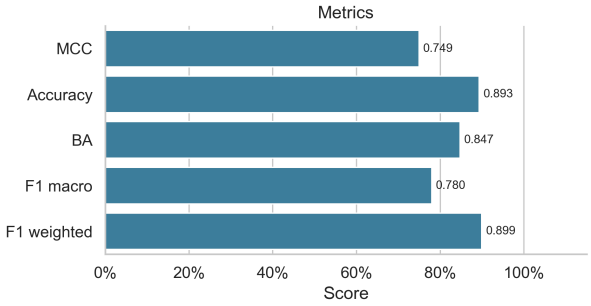
5. Practical Implications 💊

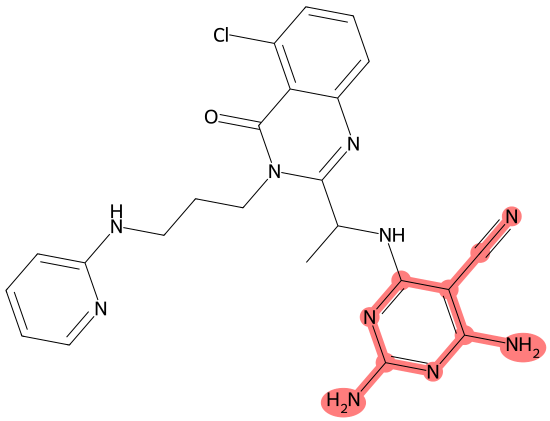
- This model can be used to **prioritize compound synthesis** and **predict selectivity** early in drug discovery.
- The MolAnchor results provide **actionable guidance** for medicinal chemists: modifying the pyrimidine core or its substituents could shift selectivity between PI3K isoforms.
- The **balanced accuracy** of 0.8471 indicates the model is robust across classes, making it suitable for **virtual screening** applications.

📌 Recommendations

1. **For PI3K-gamma selective compounds:** Retain the pyrimidine core with amine/nitrile substituents.
2. **For PI3K-delta selective compounds:** Consider modifying or removing this anchor fragment.
3. **For dual inhibitors:** The model shows lower confidence (Class 2), suggesting these compounds may require different structural features not captured in the current analysis.
4. **Further validation:** Experimental testing of compounds with/without the identified anchor fragment would validate these computational findings.






Cl
O
N
N
H
N
NH
N
N
N
NH2
N
H2N